PROJECT ON SQL

Movie Ticket Management System

Anudip AI COE Kolkata

SUBJECT: Project on SQL

TOPIC: Movie Ticket Management System

DATABASE: project

Prepared By: Sudeep Sarkar

CONTENTS

Section	Topic
1	Project Description
2	Entity Relationship Diagram
3	Code
4	Questions and Answers
5	Source Files
6	References

This documentation follows the organization of the supplied sample report and is adapted to the tables and SQL statements in the Movie Ticket Management System script.

Data note: The SQL script contains a comment indicating that CSV files were imported, but the CSV files and fixed query outputs were not supplied. For that reason, this report documents query purpose and expected result shape without inventing row counts or result values.

PROJECT DESCRIPTION

Overview

DBMS (Database Management System):

A DBMS is software used to create, store, retrieve, update, and manage data. It provides a controlled way to organize information and maintain data integrity. Examples include MySQL, PostgreSQL, and Oracle.

RDBMS (Relational Database Management System):

An RDBMS stores information in related tables made up of rows and columns. Relationships are established through keys, and SQL is used to define structures and work with records. The Movie Ticket Management System uses MySQL and the InnoDB storage engine.

SQL (Structured Query Language):

SQL is used to define database objects, insert and modify records, retrieve information, and manage transactions. This project uses commands such as CREATE, SELECT, INSERT, UPDATE, DELETE, ALTER, TRUNCATE, DROP, COMMIT, and ROLLBACK.

Project Overview

The Movie Ticket Management System is a relational database designed to organize movie information, show schedules, customer details, bookings, and individual ticket records. The database is named project and contains five main tables: movies, shows, customers, bookings, and tickets. The tables are connected with primary-key and foreign-key relationships so that a booking can be traced to a customer and a scheduled show, while ticket records can be traced to the booking that contains them.

The SQL script also includes data-validation queries, movie and show searches, booking-status summaries, ticket-revenue calculations, string-function demonstrations, and a small booking demonstration. A temporary notes table is used to practise table alteration and data-manipulation commands; it is dropped later in the script.

Objectives

- Maintain a structured movie catalogue with genre, language, and running time.
- Store show schedules, screen numbers, and base ticket prices.
- Maintain customer contact information and prevent duplicate customer email addresses.
- Connect customers and shows through booking records, with a controlled booking status.
- Store individual seat and payment-amount details for each booking.
- Use SQL queries to review booking activity, validate ticket references, and calculate confirmed-ticket revenue.

Normalization

Normalization organizes data into related tables to reduce unnecessary duplication and improve consistency. In this project, movie details are stored in movies rather than repeated in every show record; customer details are stored in customers rather than copied into every booking; and ticket details are stored separately from booking details. Foreign keys connect these records.

This structure is consistent with the basic goal of normalization, but the supplied SQL script does not include a formal functional-dependency analysis or a documented proof of 1NF, 2NF, 3NF, or BCNF. The report therefore describes the separation of entities rather than claiming that a formal normalization audit was performed.

Keys

- Primary Key: Each core table has an ID column that uniquely identifies its records: movie_id, show_id, customer_id, booking_id, and ticket_id.

- Foreign Key: shows.movie_id references movies.movie_id; bookings.customer_id references customers.customer_id; bookings.show_id references shows.show_id; tickets.booking_id references bookings.booking_id.
- Unique Key: customers.email is unique, preventing two customer records from using the same email address.
- Composite Unique Constraint: tickets has UNIQUE (booking_id, seat_number), preventing the same seat number from appearing twice within one booking.
- Candidate Key: customer email can act as a candidate key because the script requires it to be NOT NULL and UNIQUE.

Relationships

- Movies and Shows (1:M): One movie may have multiple scheduled shows; each show refers to one movie.
- Customers and Bookings (1:M): One customer may make multiple bookings; each booking refers to one customer.
- Shows and Bookings (1:M): One show may be associated with multiple bookings; each booking refers to one show.
- Bookings and Tickets (1:M): One booking may contain multiple ticket records; each ticket belongs to one booking.

Indexes

Primary-key and unique constraints create indexes that support uniqueness and lookup. The unique email constraint indexes customers.email, while the composite unique constraint covers (booking_id, seat_number) in tickets. The script does not define any separately named secondary indexes using CREATE INDEX. MySQL/InnoDB may also create supporting indexes for foreign-key constraints when required.

Transactions and ACID Properties

All five core tables use the InnoDB storage engine, which supports transactions. The script includes ROLLBACK and COMMIT commands as practice examples. However, it does not explicitly issue START TRANSACTION before the demonstration DML statements. Under MySQL default autocommit behaviour, a statement may already be committed before ROLLBACK is issued; DDL operations such as CREATE, ALTER, TRUNCATE, and DROP can also cause implicit commits.

- Atomicity: A transaction is treated as a single unit of work.
- Consistency: Constraints help keep records within defined rules.
- Isolation: Concurrent transactions are separated according to the configured isolation level.
- Durability: Committed InnoDB changes are persisted according to MySQL durability settings.

Views

A view is a virtual table defined by a stored SELECT statement. No CREATE VIEW statement appears in the supplied Movie Ticket Management SQL script, so views are not implemented in this project. The reporting queries are executed directly against the base tables.

Scope and Limitations

- The project is a database implementation and SQL practice exercise; the supplied script does not include a booking website, desktop application, or user interface.
- No payment gateway, refund workflow, or automated seat-locking process is implemented.
- The composite ticket constraint prevents duplicate seat numbers only within the same booking. It does not enforce seat exclusivity across different bookings for the same show.
- The script refers to imported CSV data, but the CSV source files are not part of the supplied materials.

ENTITY RELATIONSHIP DIAGRAM

An Entity Relationship Diagram (ERD) represents the entities in a database, the attributes stored for each entity, and the relationships between those entities. The diagram below is based on the five CREATE TABLE statements in the supplied SQL script.

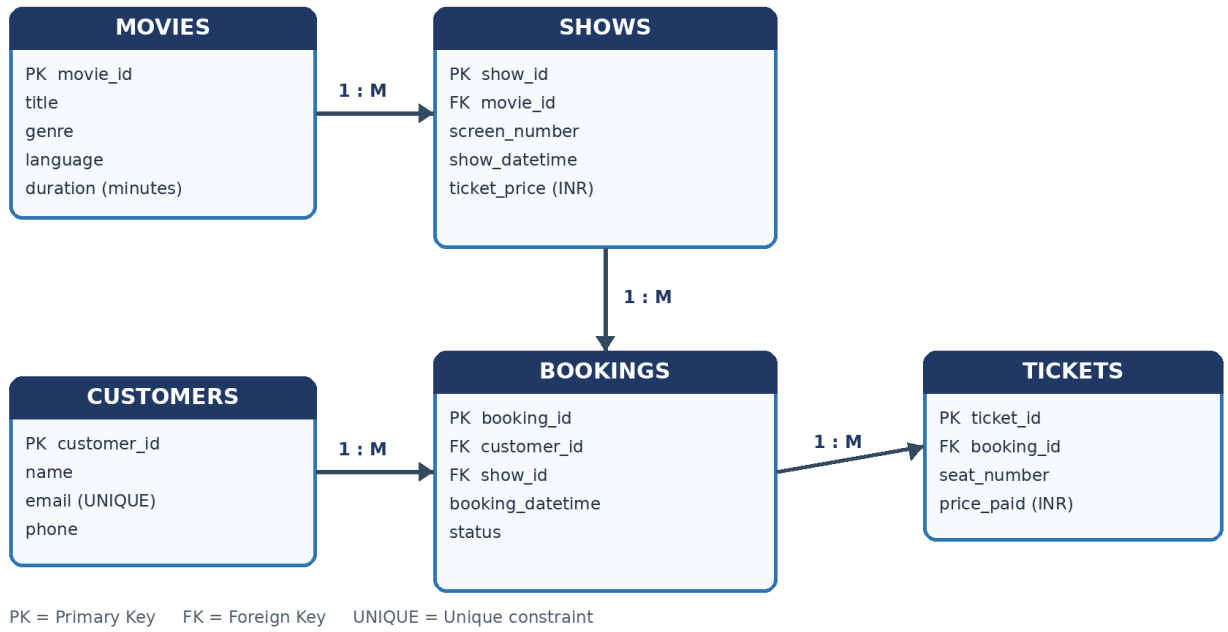


Figure 1. Entity Relationship Diagram of the Movie Ticket Management System

Entities and Attributes

Entity	Attributes
movies	movie_id INT (PK, AUTO_INCREMENT); title VARCHAR(100); genre VARCHAR(30); language VARCHAR(20); duration INT (60-240 minutes)
shows	show_id INT (PK, AUTO_INCREMENT); movie_id INT (FK); screen_number INT (1-3); show_datetime DATETIME; ticket_price DECIMAL(10,2) (INR, greater than 0)
customers	customer_id INT (PK, AUTO_INCREMENT); name VARCHAR(80); email VARCHAR(120) (UNIQUE); phone VARCHAR(20)
bookings	booking_id INT (PK, AUTO_INCREMENT); customer_id INT (FK); show_id INT (FK); booking_datetime DATETIME (default CURRENT_TIMESTAMP); status VARCHAR(20) (default Confirmed; Confirmed/Cancelled)
tickets	ticket_id INT (PK, AUTO_INCREMENT); booking_id INT (FK); seat_number VARCHAR(3) (A01-J12 pattern); price_paid DECIMAL(10,2) (INR, greater than 0); UNIQUE (booking_id, seat_number)

Key Relations

- movies to shows: Implemented by shows.movie_id referencing movies.movie_id.
- customers to bookings: Implemented by bookings.customer_id referencing customers.customer_id.
- shows to bookings: Implemented by bookings.show_id referencing shows.show_id.
- bookings to tickets: Implemented by tickets.booking_id referencing bookings.booking_id.

Important Design Notes

- Every core table is created with ENGINE=InnoDB.
- The show screen number is restricted to 1, 2, or 3, and the base ticket price must be positive.
- Booking status is restricted to Confirmed or Cancelled, with Confirmed as the default.
- Ticket seat labels are restricted by the check expression to rows A through J and seat numbers 01 through 12.
- A seat is unique only within a single booking under the current schema. The tickets table does not store show_id directly and does not contain a unique constraint that prevents the same seat from appearing in different bookings for one show.

CODE

The following code excerpts reproduce the database and table definitions from the supplied SQL script. The full original script should be run in MySQL Workbench or another MySQL client to reproduce the complete sequence.

1. Create and Select the Database

```
SHOW DATABASES;
CREATE DATABASE project;
USE project;
SELECT VERSION() AS mysql_server_version;
```

2. Movies Table

```
CREATE TABLE movies (
  movie_id INT NOT NULL AUTO_INCREMENT,
  title VARCHAR(100) NOT NULL,
  genre VARCHAR(30) NOT NULL,
  language VARCHAR(20) NOT NULL,
  duration INT NOT NULL COMMENT 'Running time in minutes',
  CONSTRAINT pk_movies PRIMARY KEY (movie_id),
  CONSTRAINT chk_movie_duration CHECK (duration BETWEEN 60 AND 240)
) ENGINE=InnoDB;
```

3. Shows Table

```
CREATE TABLE shows (
  show_id INT NOT NULL AUTO_INCREMENT,
  movie_id INT NOT NULL,
  screen_number INT NOT NULL,
  show_datetime DATETIME NOT NULL,
  ticket_price DECIMAL(10,2) NOT NULL COMMENT 'Base price per regular seat, INR',
  CONSTRAINT pk_shows PRIMARY KEY (show_id),
  CONSTRAINT fk_shows_movie FOREIGN KEY (movie_id) REFERENCES movies(movie_id),
  CONSTRAINT chk_show_screen CHECK (screen_number BETWEEN 1 AND 3),
  CONSTRAINT chk_show_price CHECK (ticket_price > 0)
) ENGINE=InnoDB;
```

4. Customers Table

```
CREATE TABLE customers (
  customer_id INT NOT NULL AUTO_INCREMENT,
  name VARCHAR(80) NOT NULL,
  email VARCHAR(120) NOT NULL,
  phone VARCHAR(20) NOT NULL,
  CONSTRAINT pk_customers PRIMARY KEY (customer_id),
  CONSTRAINT uq_customer_email UNIQUE (email)
) ENGINE=InnoDB;
```

5. Bookings Table

```
CREATE TABLE bookings (
  booking_id INT NOT NULL AUTO_INCREMENT,
  customer_id INT NOT NULL,
  show_id INT NOT NULL,
  booking_datetime DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(20) NOT NULL DEFAULT 'Confirmed',
  CONSTRAINT pk_bookings PRIMARY KEY (booking_id),
  CONSTRAINT fk_bookings_customer FOREIGN KEY (customer_id) REFERENCES customers(customer_id),
  CONSTRAINT fk_bookings_show FOREIGN KEY (show_id) REFERENCES shows(show_id),
  CONSTRAINT chk_booking_status CHECK (status IN ('Confirmed', 'Cancelled'))
) ENGINE=InnoDB;
```

6. Tickets Table

```
CREATE TABLE tickets (
  ticket_id INT NOT NULL AUTO_INCREMENT,
  booking_id INT NOT NULL,
  seat_number VARCHAR(3) NOT NULL,
  price_paid DECIMAL(10,2) NOT NULL COMMENT 'Original ticket amount after discounts, INR',
  CONSTRAINT pk_tickets PRIMARY KEY (ticket_id),
  CONSTRAINT fk_tickets_booking FOREIGN KEY (booking_id) REFERENCES bookings(booking_id),
  CONSTRAINT uq_booking_seat UNIQUE (booking_id, seat_number),
  CONSTRAINT chk_ticket_seat CHECK (seat_number REGEXP '^[A-J](0[1-9]|1[0-2])$'),
  CONSTRAINT chk_ticket_price CHECK (price_paid > 0)
) ENGINE=InnoDB;
```

7. Inspect the Database Structure

```
SHOW TABLES;
DESCRIBE movies;
DESCRIBE shows;
DESCRIBE customers;
DESCRIBE bookings;
DESCRIBE tickets;
SHOW CREATE TABLE bookings;
```

8. Data Import and Initial Validation

The SQL file contains a comment indicating that CSV data was imported into the tables. The import files are not included in the supplied materials. After importing matching records, the script checks table contents and counts with the following statements:

```
SELECT COUNT(*) FROM customers;
SELECT * FROM movies;
SELECT COUNT(*) FROM bookings;
SELECT COUNT(*) FROM tickets;
SELECT COUNT(*) FROM shows;
```

9. Booking and Ticket Reporting

```
SELECT status, COUNT(*) AS bookings
FROM bookings
GROUP BY status
ORDER BY status;

SELECT COUNT(*) AS invalid_tickets
FROM tickets t
LEFT JOIN bookings b ON b.booking_id = t.booking_id
WHERE b.booking_id IS NULL;

SELECT COUNT(*) AS confirmed_tickets,
    SUM(t.price_paid) AS net_ticket_revenue_inr
FROM tickets t
JOIN bookings b ON b.booking_id = t.booking_id
WHERE b.status = 'Confirmed';
```

10. Booking Demonstration

```
INSERT INTO movies (title, genre, language, duration)
VALUES ('SQL Demo Premiere', 'Drama', 'Hindi', 120);
SET @demo_movie_id = LAST_INSERT_ID();

INSERT INTO shows (movie_id, screen_number, show_datetime, ticket_price)
VALUES (@demo_movie_id, 1,
     TIMESTAMP(DATE(NOW() + INTERVAL 7 DAY), '18:30:00'), 220.00);
SET @demo_show_id = LAST_INSERT_ID();

SELECT @demo_movie_id AS demo_movie_id,
    @demo_show_id AS demo_show_id;
SELECT * FROM shows WHERE show_id = @demo_show_id;
```

The later fixed booking statement in the supplied script is:

```
INSERT INTO bookings
VALUES (1001, 58, 181, '2026-12-03 16:55:00', 'Confirmed');
```

Before running this fixed-ID statement, verify that customer_id 58 and show_id 181 exist and that booking_id 1001 is available. This statement uses show_id 181 rather than the @demo_show_id variable created in the preceding demonstration.

QUESTIONS AND ANSWERS

The following questions are based on queries present in the supplied project script. Each answer includes the SQL statement and the purpose of the query. Actual result rows depend on the CSV data imported into the database.

Q1. List all distinct movie genres.

```
SELECT DISTINCT genre
FROM movies
ORDER BY genre;
```

Purpose: Returns each genre only once and sorts the genre names alphabetically.

Q2. Retrieve Hindi-language movies.

```
SELECT movie_id, title, genre, language
FROM movies
WHERE language = 'Hindi';
```

Purpose: Filters the movies table to rows whose language is Hindi.

Q3. Display the five longest movies.

```
SELECT title AS movie_title, duration AS duration_minutes
FROM movies
ORDER BY duration DESC
LIMIT 5;
```

Purpose: Sorts movies by running time from longest to shortest and returns the first five.

Q4. Find movies belonging to Comedy or Animation genres.

```
SELECT title, genre
FROM movies
WHERE genre = 'Comedy' OR genre = 'Animation';
```

Purpose: Uses OR to retrieve movies matching either of the two genre conditions.

Q5. List movies that are not in the Horror genre.

```
SELECT title, genre
FROM movies
WHERE NOT genre = 'Horror';
```

Purpose: Excludes rows whose genre is Horror.

Q6. Find the first ten customers whose names begin with S.

```
SELECT customer_id, name
FROM customers
WHERE name LIKE 'S%'
ORDER BY customer_id
LIMIT 10;
```

Purpose: Uses LIKE with the percent wildcard to find names beginning with S.

Q7. Find movie titles ending with the word Stars.

```
SELECT title
FROM movies
WHERE title LIKE '%Stars';
```

Purpose: The leading percent wildcard allows any text before the ending word Stars.

Q8. Find shows with ticket prices between INR 200 and INR 250.

```
SELECT show_id, ticket_price
FROM shows
WHERE ticket_price BETWEEN 200 AND 250
ORDER BY show_id
LIMIT 10;
```

Purpose: BETWEEN includes both boundary values and returns up to ten matching shows.

Q9. List shows scheduled during July 2026.

```
SELECT show_id, show_datetime
FROM shows
WHERE show_datetime >= '2026-07-01'
  AND show_datetime < '2026-08-01'
ORDER BY show_datetime, show_id
LIMIT 10;
```

Purpose: Uses a half-open date range to include all times in July 2026, but not August 1.

Q10. Count bookings by booking status.

```
SELECT status, COUNT(*) AS bookings
FROM bookings
GROUP BY status
ORDER BY status;
```

Purpose: Groups bookings into statuses such as Confirmed and Cancelled and counts the rows in each group.

Q11. Check whether any ticket rows have no matching booking.

```
SELECT COUNT(*) AS invalid_tickets
FROM tickets t
LEFT JOIN bookings b ON b.booking_id = t.booking_id
WHERE b.booking_id IS NULL;
```

Purpose: Uses a LEFT JOIN to identify ticket rows for which no matching booking is found. The count should normally be zero when referential integrity is maintained.

Q12. Calculate the number of confirmed tickets and their net ticket revenue.

```
SELECT COUNT(*) AS confirmed_tickets,
      SUM(t.price_paid) AS net_ticket_revenue_inr
FROM tickets t
JOIN bookings b ON b.booking_id = t.booking_id
WHERE b.status = 'Confirmed';
```

Purpose: Counts ticket records and sums price_paid only for tickets associated with Confirmed bookings.

Q13. Count cancelled bookings.

```
SELECT COUNT(*) AS cancelled_bookings
FROM bookings
WHERE status = 'Cancelled';
```

Purpose: Returns the number of booking records marked Cancelled.

Q14. Count confirmed bookings for future shows.

```
SELECT COUNT(*) AS active_seats
FROM bookings b
INNER JOIN shows s ON b.show_id = s.show_id
WHERE b.status = 'Confirmed'
  AND s.show_datetime > NOW();
```

Purpose: Counts confirmed booking rows linked to shows later than the current date and time. Despite the alias active_seats, this query counts bookings, not individual ticket/seat records.

Q15. Calculate confirmed ticket revenue.

```
SELECT SUM(t.price_paid) AS revenue_inr
FROM tickets t
JOIN bookings b ON b.booking_id = t.booking_id
WHERE b.status = 'Confirmed';
```

Purpose: Sums the price_paid value for tickets in Confirmed bookings. The result is expressed in INR according to the column comment.

Q16. Apply string functions to customer details.

```
SELECT customer_id,
    UPPER(name) AS uppercase_name,
    LOWER(email) AS lowercase_email,
    CONCAT('Customer: ', name) AS customer_label,
    CHAR_LENGTH(name) AS name_length,
    SUBSTRING(name, 1, 5) AS first_five_characters,
    REPLACE(name, ' ', '_') AS username_label,
    TRIM('  SQL Project  ') AS trimmed_text
FROM customers
ORDER BY customer_id
LIMIT 5;
```

Purpose: Demonstrates uppercase, lowercase, concatenation, character length, substring, replacement, and trimming functions.

Q17. Display row counts for all five core tables in one result.

```
SELECT 'movies' AS table_name, COUNT(*) AS row_count FROM movies
UNION ALL SELECT 'shows', COUNT(*) FROM shows
UNION ALL SELECT 'customers', COUNT(*) FROM customers
UNION ALL SELECT 'bookings', COUNT(*) FROM bookings
UNION ALL SELECT 'tickets', COUNT(*) FROM tickets;
```

Purpose: Combines five count queries into one result set. UNION ALL retains every table-name row.

Q18. Insert a customer, capture its generated ID, and demonstrate modification and deletion.

```
INSERT INTO customers (name, email, phone)
VALUES ('Harsh Mehta', 'horsebhai@rmsi.com', '9087654389');
SET @demo_customer_id = LAST_INSERT_ID();
SELECT customer_id, name, email
FROM customers
WHERE customer_id = @demo_customer_id;
UPDATE customers SET name = 'Sudeep Sarkar'
WHERE name = 'Harsh Mehta';
SELECT * FROM customers WHERE customer_id = 301;
DELETE FROM customers
WHERE customer_id = @demo_customer_id;
ROLLBACK;
SELECT COUNT(*) AS demo_customer_remaining
FROM customers WHERE customer_id = @demo_customer_id;
```

Purpose: LAST_INSERT_ID() captures the generated AUTO_INCREMENT value. The remaining statements demonstrate UPDATE, SELECT, DELETE, and a subsequent ROLLBACK check.

> **Note:** The separate SELECT uses the fixed ID 301, which depends on the imported customer data. The script does not explicitly start a transaction, so ROLLBACK may not undo an autocommitted DELETE.

Q19. Create a demonstration movie and a show seven days ahead.

```
INSERT INTO movies (title, genre, language, duration)
VALUES ('SQL Demo Premiere', 'Drama', 'Hindi', 120);
SET @demo_movie_id = LAST_INSERT_ID();
INSERT INTO shows (movie_id, screen_number, show_datetime, ticket_price)
VALUES (@demo_movie_id, 1,
    TIMESTAMP(DATE(NOW() + INTERVAL 7 DAY), '18:30:00'), 220.00);
SET @demo_show_id = LAST_INSERT_ID();
```

Purpose: Creates a demonstration movie, captures its generated ID, creates a show linked to that movie, and captures the show ID.

Q20. What is the purpose of the temporary notes table exercise?

```
CREATE TABLE notes (
  note_id INT AUTO_INCREMENT PRIMARY KEY,
  note_text VARCHAR(100) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
INSERT INTO notes(note_text) VALUES ('hi');
SELECT * FROM notes;
ALTER TABLE notes ADD COLUMN category VARCHAR(20)
  NOT NULL DEFAULT 'Practice';
INSERT INTO notes (note_text)
VALUES ('Committed demonstration note'), (NULL);
SELECT note_id, COALESCE(note_text, 'No note supplied') AS note_text,
      category, created_at FROM notes ORDER BY note_id;
SELECT note_id FROM notes WHERE note_text IS NULL;
UPDATE notes SET category = 'Reviewed' WHERE note_id = 1;
SELECT * FROM notes;
DELETE FROM notes WHERE note_id = 2;
ROLLBACK;
ALTER TABLE notes DROP COLUMN category;
DESCRIBE notes;
TRUNCATE TABLE notes;
SELECT COUNT(*) AS notes_after_truncate FROM notes;
DROP TABLE notes;
COMMIT;
```

Purpose: Demonstrates table creation, inserting values (including NULL), reading rows, adding and dropping a column, COALESCE, updating, deleting, truncating, and dropping a temporary table.

> **Note:** The script includes ROLLBACK statements but does not start an explicit transaction. Under default autocommit settings, earlier DML statements may already be committed; DDL statements can also trigger implicit commits.

SOURCE FILES

File	Description
project_script.sql	The supplied MySQL script containing database creation, five core table definitions, constraints, schema-inspection statements, data-validation queries, reporting queries, a temporary notes-table exercise, and a booking demonstration.
CSV data files	The 5 inter referential CSV Files

Repository structure when assembling the final project submission:

```
Movie-Ticket-Management-System/
|-- README.md
|-- project_script.sql
|-- Movie_Ticket_Management_System_Documentation.docx
`-- data/
    |-- movies.csv
    |-- shows.csv
    |-- customers.csv
    |-- bookings.csv
    `-- tickets.csv
```

The file names shown under data are suggested placeholders; they were not present in the supplied upload.

REFERENCES

- MySQL Reference Manual: https://dev.mysql.com/doc/refman/8.0/en/
- W3Schools SQL Tutorial: https://www.w3schools.com/sql/
- SQLBolt - Interactive SQL Lessons: https://sqlbolt.com/

The database names, table definitions, constraints, SQL statements, and project-specific descriptions in this report are based on the supplied project_script(1).sql file. General DBMS, RDBMS, SQL, normalization, key, relationship, index, and transaction descriptions are introductory explanations included to match the structure of the supplied sample documentation.